

Docker Architecture

Images · Containers · Daemon · Registry

■ Analogy

- Docker is like a food truck franchise. The recipe (Dockerfile) is the blueprint, the packaged food kit (image) is what you ship, and the actual truck serving food (container) is the running instance. Any city (host OS) can run the truck — same food, same quality.

Key Terms

Docker Engine	The core runtime: daemon (dockerd) + REST API + CLI (docker)
Image	Read-only layered filesystem built from a Dockerfile
Container	A running instance of an image — isolated process with its own filesystem, network, PID
Layer	Each Dockerfile instruction adds a read-only layer; containers add a writable layer on top
Registry	Central store for images — Docker Hub, ECR, GCR, GHCR
Dockerfile	Text recipe describing how to build an image step-by-step
docker daemon	Background service (dockerd) managing containers, images, networks, volumes
containerd	Low-level container runtime that dockerd delegates to
runc	OCI-compliant runtime that actually spawns containers via Linux namespaces + cgroups
Namespace	Linux kernel feature providing isolation: PID, NET, MNT, UTS, IPC, USER

Architecture Diagram

Layer	Component	Responsibility
CLI	docker (client)	Sends REST commands to dockerd
API	Docker REST API	JSON over Unix socket / TCP
Daemon	dockerd	Orchestrates images, containers, networks, volumes
Runtime	containerd	Manages container lifecycle (start/stop/pause)
Runtime	runc	Spawns process with namespaces + cgroups
OS	Linux Kernel	Namespaces, cgroups, overlayfs

Essential Commands

Docker CLI Cheat Sheet

```
# Pull an image from Docker Hub
docker pull nginx:1.25

# Run a container (detached, port mapped, named)
docker run -d -p 8080:80 --name web nginx:1.25

# List running containers
docker ps

# Execute command inside running container
docker exec -it web bash

# View container logs
docker logs -f web

# Inspect container details (IP, mounts, env)
docker inspect web

# Stop and remove
docker stop web && docker rm web

# List images and remove unused
docker images
docker image prune -a
```

Image Layers Explained

Layer	Dockerfile Instruction	Cached?
Base OS	FROM ubuntu:22.04	Yes — shared across images
Dependencies	RUN apt-get install -y java	Yes — unless instruction changes
App code	COPY target/app.jar /app.jar	Invalidated when jar changes
Config	ENV JAVA_OPTS=-Xmx512m	Yes — unless value changes
Writable	(container layer)	No — discarded on container removal

■ Real-World: Spotify

- Spotify containerized its microservices using Docker to eliminate 'works on my machine' problems across 3000+ engineers. Each service ships as a Docker image tested identically in dev, CI, and production — reducing environment-related incidents by over 60%.